Lab 4 - Frontend and Backend

Student Sheet Duration: 100 minutes

AI1220 — Software, Web & Mobile Engineering: Code without Limits

Start here: install the desktop app

1. Visit `https://openai.com/codex/` and install the ChatGPT/Codex desktop app for your computer. Open it and sign in with your ChatGPT account. The official desktop guide has download and setup instructions.
2. Download and extract `lab04_files.zip`. Copy its contents into `lab04/` inside your existing private `ai1220` repository. Git is already configured; keep using that repository.
3. In the desktop app, select **Codex**. Add a local project with `ai1220/lab04` as its folder. Use **Edit project > Add folder** if needed, and **Make primary** to set this as the working folder. Start a new chat in **Local** mode.
4. Ask Codex: "Show the working folder and Git repository root. Explain how backend.py and index.html work together. I know Python but I am new to web development. Do not change files yet." Check that the folder is `lab04` and the repository is `ai1220`.

Use this chat for the exercises. Give Codex one task at a time, inspect its edits, and check the result yourself. Exercise 1 introduces a simple commit through Codex.

If account access or a usage limit blocks you, tell the instructor and continue with the supplied files and guided explanation. No API key or purchase is required for the local playlist.

Goal

A small music club needs a page where someone can view a playlist and add a song's title and artist. Complete the two marked functions in the starter, then show that the browser and Python server agree on what was added. Each student works in their own repository. Use invented song data.

Setup

You need Python 3.10 or newer and a browser. Keep `backend.py` and `index.html` together in the working folder you selected above.

Open a terminal in `lab04/`, then run:

```
python3 backend.py
```

On Windows, use `py backend.py`. Open `http://127.0.0.1:8000/` in your browser. The starter displays **First Light / Demo Band**; adding is unfinished. Keep the terminal running. No extra Python packages are needed. Stop with Ctrl+C. After Python edits, restart the server; after HTML edits, refresh the page. Restarting restores the original one-song playlist.

Key concepts

Frontend / backend	The browser page handles interaction; the Python server owns the playlist.
HTTP request / response	A message to the server and its reply. GET reads; POST asks to add.
JSON / API contract	JSON represents data as text; the contract fixes paths, fields, and replies.
Commit / push	A commit records a snapshot in local Git history; a push sends commits to GitHub.

The supplied contract

The same server supplies the page at `/` and the data at `/songs`. A path identifies what is requested. A status code describes the result: 200 means OK, 201 means created, and 400 means invalid input. JSON requests and replies use `Content-Type: application/json`.

Request	Required result
`GET /songs`	200 and a JSON list of all stored songs, in insertion order. No request body.
`POST /songs`	Send a JSON object with `title` and `artist`. Return 201 and the newly stored song, including its server-assigned integer `id`.
Invalid addition	Return 400 and `{"error": "a helpful message"}`. Leave the playlist and the next ID unchanged.

Both fields must be strings, nonempty after surrounding whitespace is removed, and at most 80 characters after trimming. Store the trimmed values. Ignore extra fields. IDs start at 1 and increase by one for each accepted addition. Duplicate titles are allowed. Data exists only until the server stops. The supplied server handles malformed JSON, non-object bodies, and unknown paths.

From a fresh start, the first successful addition has this shape:

```
POST /songs          body: {"title":"Blue Sky","artist":"Test Duo"}
201 Created         body: {"id":2,"title":"Blue Sky","artist":"Test Duo"}
```

The next `GET /songs` must contain both the seed song and this new song.

Exercise 1: Explore and make a commit — 15%

Run the starter and read Codex's explanation. Ask Codex to enter your name in `REPORT.md`, then inspect the changed file in the app. Send this instruction:

"Check Git status. In my existing ai1220 repository, commit only this lab's REPORT.md with the message 'Start lab04 report'. Preserve unrelated changes and staging. Show the commit hash and files included. Do not push yet."

Verify that the commit contains only `lab04/REPORT.md`. In the report, record its hash, what was saved, and which file owns the playlist and which displays it. Explain how the page gets the initial song. Record any Codex access issue and complete the equivalent steps with instructor support.

Exercise 2: Complete the Python backend — 35%

Complete only the marked `create_song(payload)` function in `backend.py`. It must satisfy the contract above for any valid title and artist. Its docstring describes how to report invalid input to the supplied server. Keep the existing HTTP setup. Check both an accepted and a rejected request before moving on.

Exercise 3: Connect the frontend — 30%

Change the visible page heading to **My playlist**. Complete the marked `sendSong(title, artist)` function in `index.html`. It must send the form values to `POST /songs` as JSON and return the response using the supplied `requestJSON` helper. Ask Codex to explain how its changes match the contract.

The form handler and list display are already supplied. An accepted addition must appear in the list and clear the inputs. A rejected addition must show an error and retain the inputs so the user can correct them. The page must display what the server actually stores.

Exercise 4: Verify and explain — 20%

Run the checks on page 3. In `REPORT.md`, record actual observations, one successful request and response, and one failed request and response. Describe one code change you reviewed and why its observed result agrees with the contract. Be ready to explain the two completed functions.

How to check your work

Use the browser for the user journey and a second terminal for direct requests. The terminal requests below work in macOS/Linux shells; on Windows, use Git Bash. `curl` sends an HTTP request; `-i` shows the reply status and headers, `-H` adds a header, and `-d` supplies a POST body.

1. Restart the server. The page must show only **First Light / Demo Band**. The following request must return that one song with `id: 1`:

```
curl -i http://127.0.0.1:8000/songs
```

2. Add **Blue Sky / Test Duo** through the form. It must appear once; the fields clear. Refresh the page: both songs remain. Add another invented song to check that your code works with different values.

3. Add a song directly. Expect 201, trimmed values, and the next unused ID. Refresh the page and confirm that it appears:

```
curl -i http://127.0.0.1:8000/songs \
  -H 'Content-Type: application/json' \
  -d '{"title":"  Quiet Road  ","artist":"Sample Artist"}'
```

4. Send invalid input directly. Expect 400, an error message, and no new song:

```
curl -i http://127.0.0.1:8000/songs \
  -H 'Content-Type: application/json' \
  -d '{"title":"   ","artist":"Sample Artist"}'
```

   Also check a missing artist, a numeric title, and a title of 81 characters. A valid 80-character title must succeed. Rejected requests must not consume an ID.

5. In the form, submit spaces as the title and a nonempty artist. The error must be visible, the inputs must remain, and the list must stay unchanged. Then correct the title and submit successfully. Use Tab and Enter to check keyboard access.

6. Open the browser's Developer Tools, then **Network**. Add a song. Inspect `POST /songs`: its payload, status 201, and JSON response. Inspect the following `GET /songs`. Finally, restart the server and refresh: only the seed song remains.

Troubleshooting

Cannot open the page	Keep the server terminal open; use the exact HTTP URL on page 1.
Port 8000 is in use	Stop your earlier lab server with Ctrl+C. Ask for help if it is another process.
Changes are not visible	Save first; restart Python or refresh HTML as appropriate.
"Not implemented"	This is expected until the relevant marked function is completed.

Deliverables

Keep `backend.py`, `index.html`, `REPORT.md`, `README.md`, and `.gitignore` in `ai1220/lab04/`. Ask Codex to review and commit the remaining changes in those files with the message `Complete lab04 playlist`, preserving other work. Then use your existing terminal workflow to push from the repository root:

```
git push
```

Push before the session ends and verify the files on GitHub.

Optional stretch (ungraded): Display the number of songs and check that it remains correct after additions and refreshes.